

Adopting a Subagent Framework (Replacing Modules with Subagents)

Replacing Modules with Named Subagents

In the new design, each major **module** from the previous architecture is replaced by a specialised **subagent** with a clear role. This aligns with the subagent definitions in *SUBAGENTS.md* and provides a more modular, scalable structure. The table below maps the original modules to their corresponding subagents and roles:

- **Context Module** → **Context-Manager Subagent** – Handles loading and writing of context files and compaction of history. This agent reads all relevant context files (plans, tasks, logs, etc.), produces concise context digests, and triggers compression when needed ¹.
- **Analysis Module** → **Analysis-Agent** – Responsible for mapping out the problem domain. It identifies key entities (tasks, concepts, components, etc.) and relationships, then updates the Tree-of-Thought (ToT) diagram (stored in *TOT.md*) accordingly ². This agent essentially builds the “meta tree” of the problem by performing dependency analysis and syncing findings to the knowledge graph.
- **Research Module** → **Research-Agent** – Conducts in-depth information gathering. It performs internal and external research (e.g. web searches, documentation lookup), cross-validates facts from multiple sources, and compiles findings with citations into *RESEARCH.md* ³. This agent is triggered whenever information gaps or uncertainties are identified, ensuring a **research-first** approach before solution generation.
- **Brainstorm Module** → **Brainstorm-Agent** – Generates and evaluates ideas or solution components. After research is synthesised, this agent produces a list of possible approaches, modules, or features and scores each for feasibility, impact, and elegance ⁴. The top-ranked ideas are recorded in *BRAINSTORM.md*, providing a creative exploration of options before planning.
- **Planning Module** → **Planning-Task-Agent** – Synthesises all insights (context, research, brainstorm results) into a concrete project plan ⁵. It creates a structured roadmap, breaking the work into phases and tasks. This agent updates the persistent plan (*planning.md*) and task list (*todo.md*) with numbered steps, and may consult virtual expert subagents (like a requirements analyst or architect persona) to refine the plan.
- **Execution Module** → **Execution-Agent** – Handles implementation of tasks iteratively. Instead of the monolithic execution module, this subagent carries out each task in the to-do list by invoking the appropriate tools and domain-specific developer agents ⁶ ⁷. It writes or modifies code, runs tests, and integrates results, adhering to design and security guidelines at each step. The Execution-Agent ensures that each code or content change is done according to the plan and project conventions.
- **Review Module** → **Review-Reflection-Agent** – Performs thorough review and critique of the outputs. This subagent simulates a panel of experts to examine the work (for example, checking requirements coverage, code quality, UX design, performance, security) ⁸. It captures feedback and lessons learned as **reflections**, possibly spawning new fix or improvement tasks. The review agent’s output is recorded as reflection notes or bug reports, feeding back into planning if needed.

- **Delivery Module** → **Delivery-Agent** – In charge of final validation and packaging of results. This agent verifies all tasks are completed and all acceptance criteria are met ⁹. It then packages deliverables (files, documentation, final messages) and presents the results to the user in a clear format. The Delivery-Agent uses tools (e.g. syncing files) to attach relevant artifacts (reports, plans, code) and generates the final user-facing message.

Each subagent has a dedicated purpose and is only invoked when its particular expertise is required, keeping the system modular and focused. This one-to-one replacement of modules with subagents preserves the **clear boundaries and responsibilities** of the original modular design ¹⁰ while leveraging the benefits of agent specialisation and isolation.

Main Orchestrator (“Claude”) and Invocation Workflow

The **main orchestrator agent**, referred to as *Claude* in this design, serves as the brain of the system that coordinates all subagents. Claude **never solves tasks directly** and does not contain the business logic of each phase; instead, it **orchestrates a swarm of subagents** according to a defined lifecycle ¹¹. It reads the user’s request and the current context, then decides which subagent (or sequence of subagents) should handle the next step. The orchestrator’s responsibilities include maintaining the global loop, managing task queues, and ensuring context is properly shared between agents.

A key addition to the architecture is a **Workflow Gateway** at the decision point of each task ¹². When a new user request or task comes in, Claude first checks if it matches any predefined **workflow** in the workflow library (*WORKFLOWS.md*). Workflows are essentially scripts that chain multiple subagent calls to handle common complex scenarios ¹³. For example, a “New Project” workflow might automatically invoke a sequence like: *new-project-setup-agent* → *context-manager* → *planning-agent* → *research-agent* → *delivery-agent* ¹⁴. If a workflow trigger matches (e.g. user explicitly asks for a new project, or a complex multi-step task), Claude will follow that **predefined subagent sequence**. This gateway approach ensures consistent, repeatable processes for known scenarios and reduces cognitive load on the orchestrator by leveraging declarative workflows ¹⁵.

If no workflow is appropriate, Claude falls back to a dynamic decision: it can call the **Invocation-Chain-Generator** subagent to recommend the optimal chain of subagents for the task at hand ¹⁶. The Invocation-Chain-Generator will analyze the high-level objective and the current knowledge graph state, then propose a sequence of agent calls (and tools) that would efficiently achieve the goal ¹⁶. Claude can use this suggestion to invoke a series of subagents one by one. In simple cases (for example, a straightforward request that only needs one type of expertise), Claude may directly invoke a single appropriate subagent (this is used for the “rapid task” workflow, where a quick safety check is done then one agent solves the task) ¹⁷.

Clear invocation paths are defined for each type of situation: either follow a known workflow or choose a single subagent/chain. Claude logs a **ModuleSelection** (or rather *AgentSelection*) event whenever it picks a subagent or workflow, recording the rationale for transparency ¹⁸. This mechanism guarantees that for each user objective, there is a clear and explainable sequence of agent calls rather than an arbitrary chain. It blends **structured workflows** with flexible single-agent calls, achieving a balance between deterministic behavior and adaptability ¹⁵ ¹⁹.

Throughout this process, Claude adheres to an iterative **think-act-observe** loop (inspired by ReAct). In each cycle, Claude: (1) **Thinks** – analyzes the current state and decides on an action (which subagent/tool to invoke next) ¹⁸; (2) **Acts** – invokes the chosen subagent with the relevant context (this subagent may perform a tool call, code generation, etc.) ²⁰; (3) **Observes** – waits for the subagent’s result and then interprets the outcome, logging an Observation event ²¹. These steps repeat, possibly looping back to the workflow gateway or into further reasoning, until the task is complete ²². This *atomic process* loop ensures the system remains responsive to new information: if a subagent’s observation reveals new tasks or information (for example, research finding new requirements), Claude can adjust the plan or even revisit earlier phases as needed. It will log **PlanUpdate** events whenever the plan or to-do list changes on the fly ²³. Importantly, Claude employs **Reflexion** (self-critique) at key intervals – essentially simulating a pause to double-check its own reasoning. This is achieved by leveraging the Review-Reflection agent on itself or by internal checks, ensuring any potential flaws or oversights are caught before proceeding ²⁴.

In summary, Claude is the **coordinator** that manages when and how each subagent is invoked. It uses the workflow gateway and invocation chain logic to chart a path for each user request, then uses a ReAct-style loop to execute that path step by step. The result is a controlled yet flexible orchestration of subagents, with Claude *“never creating subagents itself but coordinating existing ones efficiently”* ²⁵ ²⁶.

Integrating Cross-Cutting Concerns into Subagents

The previous design included several **cross-cutting modules** (logging, guardrails, memory, design, testing, security, etc.) that spanned all phases. In the subagent framework, these concerns are not separate free-floating modules; instead, they are **integrated into the responsibilities of specific subagents or the coordinator**. This reduces overlap and ambiguity ²⁷, since each concern now has a clear owner (or agent) that handles it across the workflow:

- **Safety & Guardrails:** All safety, compliance, and content sanitisation duties previously in a *guardrails_module* are now handled by the **Safety-Compliance-Agent**. This specialised subagent is invoked whenever external information is fetched or a potentially risky action is about to be executed ²⁸. It verifies information against multiple sources, sanitises inputs/outputs to remove any disallowed content, confirms with the user before performing side-effect actions, and ensures regulatory compliance (e.g. privacy laws) ²⁸ ²⁹. By having a dedicated safety agent, the system enforces guardrails consistently whenever needed, rather than scattering checks throughout the code.
- **Memory & Knowledge Management:** Instead of a generic memory module, the system uses two cooperating subagents to manage long-term knowledge. The **Memory-Manager** ensures that important entities, relations, and observations are persisted to the knowledge graph (acting as the “memory” of the system) ³⁰. It creates or updates nodes for tasks, files, requirements, etc., and logs observations like metrics or test results into the graph with timestamps and provenance ³¹. The **Graph-Manager** complements this by handling semantic queries and retrieval from the knowledge graph ³². For example, during context gathering or analysis, the Graph-Manager can be invoked to find relevant prior information or dependencies from the graph (essentially performing the role of the old *graph_module* for knowledge retrieval) ³². Together, these ensure that context from earlier in the project or previous sessions is not lost and can be recalled when needed, without overloading the prompt.
- **Logging:** There is no longer a standalone logging module; instead, the **coordinator (Claude)** itself takes on structured logging of events as an implicit function. At each significant step, Claude

appends an entry to the chronological event log (*event-stream.md*)³³. These events include phase transitions, subagent selections, actions taken (tool invocations), observations/results, updates to plans, and any errors³⁴³⁵. The logging follows a consistent format (e.g., **UserMessage**, **Action**, **Observation**, **PlanUpdate**, **Error**, **Delivery** events) so that the chronology of the agent's reasoning and actions is transparent³⁶³⁷. By integrating logging into the core loop, we ensure every subagent's contribution is recorded without needing a separate logging agent.

- **Context Engineering:** The task of managing the context window – choosing what information to carry forward or summarise – is woven into the **Context-Manager** agent's duties and the orchestrator's guiding principles. The context-manager prepares “quick” or “full” digests of context for subagents¹, meaning it applies the *write/select/compress/isolate* strategies that were in the old *context_engineering_module*³⁸. For instance, it might summarise older event history when the context grows too large (auto-compaction) and isolate each subagent's scratchpad so that intermediate reasoning doesn't leak into another agent's context³⁹. The orchestrator's **Context Engineering** principle⁴⁰ also ensures that only the most relevant snippets of context and memory are included when invoking a subagent, preventing context bloat and hallucinations. In short, context management is now an active process performed by the context-manager (with help from memory/graph managers), rather than a passive module.
- **Design & Accessibility:** The design guidelines (UI/UX consistency, accessibility standards, etc.) previously noted in a design module⁴¹ are enforced by a domain-specific subagent called **Design-Accessibility-Agent**. This agent can be called during execution or review phases to ensure front-end components, styling, and content meet the design system and accessibility requirements. In practice, the Execution-Agent will invoke the design-accessibility helper when implementing UI tasks, and the Review phase might include a design/UX critique via this agent⁴². By encapsulating design checks in a subagent, the system ensures that design compliance is verified whenever relevant changes are made.
- **Testing & QA:** Similarly, testing is handled by the **Testing-QA-Agent** instead of a generic testing module⁴³. This subagent is responsible for running test suites (unit tests, integration tests, etc.), performing quality assurance checks, and reporting results. The Execution-Agent calls the testing agent after generating or modifying code to verify correctness, and in the Review phase the testing agent may run additional regression or performance tests⁴⁴⁴⁵. All test outcomes are fed back as observations and, if failures occur, new tasks may be created to fix issues.
- **Security:** Security concerns (such as checking for vulnerabilities, secrets exposure, compliance with privacy rules) are largely covered by the **Safety-Compliance-Agent** in this framework. The execution agent is instructed to **follow security guidelines** during implementation⁶, and the safety agent will specifically flag any dangerous patterns (e.g., hard-coded credentials or injection risks) either during code execution or review. In a complex scenario, we could introduce a dedicated “Security-Audit-Agent” (domain expert) during the Review phase, but in the current design the safety/compliance agent and the general review process handle most of these checks⁴⁶.
- **Documentation & Logging (Development):** Documentation tasks (like ensuring code comments or user docs are updated) and code quality checks are handled by domain-specific agents such as a **Documentation-Agent** or a **Quality-Review-Agent** during execution and review. For example, the Execution-Agent might delegate documentation updates to a documentation subagent (if complex), and the Review-Reflection agent might include a *quality-review-documentation-agent* to proofread and check the completeness of documentation⁴⁷. This approach localises documentation concerns to relevant points in the workflow, rather than needing a continuous documentation module.

By integrating these cross-cutting concerns into subagents, the architecture avoids the “overlapping modules” problem where similar tasks might be handled in multiple places²⁷. Each subagent now has a

well-defined sphere of responsibility that includes any related cross-cutting issues. For instance, anything related to safety or compliance is funneled through the safety agent, and anything involving persistent memory goes through the memory/graph managers. This not only simplifies the design but also makes it clearer which agent to invoke for a given concern, improving maintainability.

Phase-by-Phase Process with Explicit Subagent Roles

The overall **lifecycle** of the agent remains divided into clear phases (Context Gathering, Analysis, Research, Brainstorming, Planning, Execution, Review, Delivery), but now each phase description explicitly references which subagents are involved and how they interoperate. Here is a walkthrough of each phase in the subagent-based framework, highlighting the agents and their interactions:

- **1. Context Gathering:** The goal of this initial phase is to understand the user's request and assemble all relevant information within the context window ⁴⁸. At session start or when a new task begins, Claude invokes the **Context-Manager** to load the persistent context files (prior plans, to-do lists, event logs, conventions) and any recent artifacts (research notes, brainstorm results, etc.) ⁴⁹. The context-manager produces a condensed summary of the current state, ensuring only the most pertinent details (related to the user query) are highlighted. Claude also calls on the **Graph-Manager** to query the knowledge graph for any entities or past projects related to the user's request (for example, if the user is asking for a feature that has dependencies, the graph-manager will retrieve those links) ⁵⁰. The results of context loading and graph queries are then summarised in an internal context summary or written to *RESEARCH.md* as an initial context overview ⁵¹. During this phase, if any critical detail is missing, Claude may ask a clarifying question to the user (avoiding unnecessary queries) ⁵². The **Safety-Compliance-Agent** is implicitly involved here as well: if any user-provided data or loaded context contains potentially unsafe or disallowed content, the safety agent sanitises it before it's used by others. The output of Context Gathering is a clear problem statement (Claude internally re-states the user's goals) and a curated context ready for downstream agents, plus a log of a **UserMessage** event to record the user's intent ⁵³. (If the goals or context are still unclear, this phase can loop, possibly triggering additional context loads or questions until clarity is achieved ⁵⁴.)
- **2. Analysis (Building the "Meta" Knowledge Map):** Once the context is set, Claude moves to analysis. Here the **Analysis-Agent** takes center stage. Claude invokes this agent to identify key **entities, components, and relationships** from the information gathered ⁵⁵. For example, it will parse the problem and pick out entities like "subagents", "workflow gateway", "knowledge graph" (as in this project) and map how they relate (e.g., subagents are orchestrated by the gateway; subagents update the knowledge graph; etc.). The analysis-agent then constructs a visual **Tree-of-Thought (ToT)** or a graph-of-thought diagram that hierarchically or graphically represents the problem structure ⁵⁶. In practice, this means updating the *TOT.md* file (often using a Mermaid diagram to show nodes and connections) ⁵⁷ – essentially creating a "meta tree" of the solution space. The agent also ensures any new entities or relations discovered are added to the knowledge graph via the memory/graph managers. Claude logs an **Analysis** event (as a form of knowledge capture) for the new structured understanding recorded. This Analysis phase may be revisited (or the ToT updated) if later phases uncover new entities or relationships that were not originally mapped ⁵⁸. At this point, the system has a shared mental model of the problem, which will guide subsequent research and planning.

- **3. Research & Synthesis:** With the entity map in hand, the next step is to fill in knowledge gaps about those entities or the domain. Claude invokes the **Research-Agent** to perform targeted investigation ⁵⁹. This agent uses the browser and other tools to search authoritative sources: for example, reading documentation, academic papers, or credible websites relevant to each key component in the ToT map. As the research-agent gathers facts, the **Safety-Compliance-Agent** is used to double-check external information (especially if code or instructions are fetched from the internet – they must be verified and sanitized) ⁶⁰. All useful findings are distilled into notes in *RESEARCH.md*, with proper citations to sources ⁶¹. The research-agent prioritises information by relevance and source trustworthiness ⁶², possibly using the knowledge graph to tag which entity each fact belongs to and how important it is (for instance, linking a source as evidence supporting a specific module design in the knowledge graph) ⁶². The result of this phase is a coherent research digest that the system can rely on for factual accuracy. If contradictions or knowledge gaps persist, the agent may loop: performing additional searches or even returning to context gathering to find missing pieces ⁶³. Once the research-agent finishes, we have an evidence-based foundation for solution development. This completes the “**research-first**” mandate of the system, ensuring the next steps are grounded in facts ⁶⁴.
- **4. Brainstorm & Evaluation:** After gathering knowledge, the **Brainstorm-Agent** is invoked to generate creative solutions or components ⁶⁵. Claude triggers this agent to come up with a breadth of ideas – for example, different architectural approaches, optional features, or improvements to incorporate. The brainstorm-agent doesn’t work blindly; it leverages the research synthesis and the ToT map as inspiration (so that ideas are relevant to the identified entities and user goals) ⁴. It might propose, say, “introduce a guardrails subagent for safety” or “use a workflow for known patterns” as ideas, each of which is then **evaluated and scored**. The agent assigns a score (e.g. 1–10) to each idea along with a rationale for its feasibility and impact ⁶⁶. These scores and notes are recorded in *BRAINSTORM.md*. Claude (as orchestrator) can then pick the top-ranked components or approaches from this list to include in the final plan ⁶⁷. The Brainstorm phase encourages **divergent thinking** followed by **convergence** on the best ideas. If later in the process new ideas emerge or some turn out to be unfeasible, the brainstorm document can be revisited and updated, ensuring the design space stays flexible ⁶⁸.
- **5. Planning:** Now comes the creation of an actionable plan. Claude calls the **Planning-Task-Agent** to synthesize everything learned into a concrete execution plan ⁶⁹. The planning agent takes the top ideas from brainstorming, the constraints and goals from context, and the factual knowledge from research, then formulates a step-by-step **roadmap**. This includes breaking the project into phases or components (for example, “setup environment”, “implement feature X”, “write tests”, “perform review”) and generating a list of tasks in logical order ⁷⁰. Each task is written to the to-do list (*todo.md*) along with any dependencies or prerequisites noted. The planning agent also updates *planning.md* with a high-level narrative of the plan, current status of phases, and design decisions. Crucially, the planning agent engages a “virtual panel of experts” to refine the plan ⁷¹. This means it may internally simulate asking different expert subagents – e.g. a **Requirements-Agent** to ensure all user needs are met, an **Architecture-Agent** to check system design soundness, a **Performance expert** for efficiency concerns, etc. – and incorporate their feedback. (These expert perspectives can be thought of as roles the planning-agent takes on or additional subagents it calls; in the architecture they might correspond to domain-specific review agents or simply reasoning steps.) The feedback is recorded in *planning.md* (almost like an internal review of the plan), and the plan is adjusted until it’s robust ⁷¹. The outcome is a detailed, peer-reviewed plan and an initial task

backlog. Claude logs **PlanUpdate** events for the creation of tasks and any changes made during planning ⁷². If at any point requirements change or new information comes in, this phase can repeat to update the plan and task list (with changes synced to the knowledge graph via the memory-manager). Now the system is ready to execute.

- **6. Execution:** Claude proceeds to execute the tasks iteratively, guided by the plan. For each task in *todo.md*, the orchestrator will invoke the **Execution-Agent** to carry it out ⁷³. The execution agent reads the current task details and decides which *tool* or *domain-specific subagent* is needed. It may itself call on specialised developer agents such as a **Developer-Frontend-Agent**, **Developer-Backend-Agent**, **Testing-QA-Agent**, **Design-Accessibility-Agent**, etc., depending on the nature of the task ⁴² ⁷. For example, if the task is “Implement the UI for feature X,” the execution agent might specifically use the frontend subagent to generate React components; if the task is “Run load tests,” it will use the testing-QA subagent, and so forth. Each concrete action (like running code, editing a file, or calling an external API) is logged as an **Action** event, and the result (success, failure, output data) is logged as an **Observation** event ⁷⁴. The safety agent is on standby during execution as well – e.g., if the code needs to run a potentially dangerous operation or if output needs sanitising, the safety-compliance checks will be applied before proceeding. After each task is completed, the Execution-Agent marks it off in *todo.md* and updates any relevant artifacts or context (for instance, updating the knowledge graph that a certain component is now implemented, or writing any new findings to RESEARCH.md). Claude logs a PlanUpdate when tasks are completed or if new tasks emerge (sometimes executing a task can reveal a follow-up task, which the planning agent may add to the list) ⁷⁵. This phase is essentially an iterative loop within itself – each task cycles through thought (deciding how to execute), action, observation, until all tasks in the current phase are done. If an error is encountered (e.g., tests fail or a bug is found), the Execution-Agent will log an **Error** event and possibly retry or create a remediation task ⁷⁶. The orchestrator may even pause execution and bounce back to research or planning if the problem requires it (for example, if an unforeseen technical challenge arises, it might trigger a quick research sub-task, then resume execution). Execution continues until the to-do list for the project (or the current milestone) is fully checked off.
- **7. Review & Reflection:** After execution, the system enters a critical evaluation phase. Claude invokes the **Review-Reflection-Agent** to simulate an **expert review panel** and perform self-reflection on the delivered outputs ⁷⁷. The review agent will take on (or coordinate) multiple expert personas: requirements reviewers to ensure all user needs are met, architects to examine the code structure, UX designers to review interfaces, security auditors, performance analysts, etc. ⁷⁷ ⁷⁸. It aggregates feedback on the work’s quality and identifies any shortcomings or potential improvements. For example, the review might reveal that some edge-case is not handled or a security best practice was missed. All these findings are logged, with severe issues clearly marked. The agent (or Claude itself as part of reflexion) will then decide on follow-up actions for any critical problems – this could mean creating new tasks in the to-do list (like “fix security vulnerability in module Y”) or if minor, simply noting them for future improvements ⁷⁹ ⁸⁰. Each insight or lesson learned is captured as a **KnowledgeCapture** or **Analysis** event in the log, ensuring the system’s knowledge graph grows with these reflections. This phase essentially serves as a **critique** loop: the system is checking and criticizing its own work, which greatly reduces the chance of delivering flawed results. If major issues are found, the workflow might route back into an earlier phase (planning or execution) to address them, and then another review cycle runs. This repeats until the panel (and internal self-critique) are satisfied that the outputs meet the quality bar and align with the

user's goals ⁸¹ . By the end of reflection, the deliverables should be polished and all critical feedback resolved.

- **8. Delivery:** Finally, Claude uses the **Delivery-Agent** to wrap up the project and present results ⁸² . The delivery agent double-checks that every task in *todo.md* is marked complete and that all artifacts and context files are up-to-date and consistent ⁸³ . It gathers all deliverable documents – for this orchestrator project, that could include the updated *CLAUDE.md* (the agent prompt), *CLAUDE_PROCESS.md* (the process definition), and artifact files like the *TOT.md* diagram, *RESEARCH.md* summary, *BRAINSTORM.md*, and updated *planning.md* and *todo.md* ⁸⁴ . Using the synchronization tools available, the delivery agent attaches these files for the user. In the final message to the user, the agent provides a clear summary of what was accomplished, highlights key design decisions, and ensures no internal reasoning or confidential info is leaked (only the polished prompts and plans are shared) ⁸⁵ . A **Delivery** event is logged in the event stream to mark completion. At this point Claude transitions to a **standby** state, awaiting further instructions ³⁷ ⁸⁶ . The entire process is now complete: the user receives the improved system specifications or whatever outputs were requested, and all intermediate steps have been documented.

Throughout all these phases, the orchestrator and subagents maintain a strict adherence to the guiding principles (research-first, context management, knowledge graph usage, iterative refinement, etc.) ⁶⁴ ⁸⁷ . Every phase transition is logged (PhaseChange events) and the knowledge graph is updated continuously so that even if the loop returns to an earlier phase, it does so with more information than before. The result is a **cohesive hierarchy** of reasoning: from high-level goal understanding (the meta level context and ToT map), down to low-level **atomic actions** (think-act-observe cycles in execution), incorporating **simulation** of expert feedback (review/reflection agent), and rigorous **critique** of the outcomes (self-reflection and improvements), all through explicit subagent collaborations.

In summary, converting the architecture to a subagent framework has **clarified the invocation paths and responsibilities** for each step of the process. The main orchestrator “Claude” now cleanly delegates tasks to named subagents (context-manager, research-agent, etc.) instead of monolithic modules, and uses the workflow gateway to chain these subagent calls in a logical sequence ⁸⁸ . Cross-cutting concerns like safety, memory, and logging are woven into these agents rather than handled by separate modules, reducing overlap. Each phase of the lifecycle explicitly calls out which agent is doing what, making the process more transparent and maintainable. This redesign preserves the structured, phase-by-phase approach of the original modular design ¹⁰ , but with the added flexibility and clarity of an agent-oriented architecture. All of these changes together result in a robust, **multi-agent orchestration system** that is easier to understand, extend, and ensure correctness for complex tasks.

¹ ² ³ ⁴ ⁵ ⁶ ⁷ ⁸ ⁹ ¹⁶ ²⁸ ³⁰ ³² SUBAGENTS.md

file:///file-BTQPEGKwQRaCjt7HHqHPT

¹⁰ ³⁸ BRAINSTORM.md

file:///file-5c575pLKRYAy4hajfsQPD3

¹¹ ¹⁸ ²⁰ ²¹ ²² ²³ ²⁴ ²⁵ ²⁶ ²⁹ ³¹ ³³ ³⁴ ³⁵ ³⁶ ³⁷ ³⁹ ⁴⁰ ⁴¹ ⁴² ⁴³ ⁴⁴ ⁴⁵ ⁴⁶ ⁴⁷ ⁶⁴ ⁸⁷

CLAUDE.md

file:///file-YPwtq8BwWF5mYqDvq1ucsW

12 15 19 88 **PROCESS_MAPS.md**

file:///file-AuCeYGBN7u6QuopXykGE1Z

13 14 17 **WORKFLOWS.md**

file:///file-YVw3oJydYyrbgjxQHtiyb

27 78 **Analyze instructions comprehensively.pdf**

file:///file-PEPoqsSSH669XTzJKuvH2

48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 65 66 67 68 69 70 71 72 73 74 75 76 77 79

80 81 82 83 84 85 86 **CLAUDE_PROCESS.md**

file:///file-94SLwPEkFGFiqRqwroH3uY